

Relatorio do trial

PARTICIPANTE

Luis Henrique Gonçalves Barbosa

KATA

Tarifador de Hospedagem e Pousada

TRATAMENTO

Sem assistente de IA

TEMPO FINAL

25:30

TEMPO ESGOTADO

Nao

INICIO

16/09/2026, 13:37:44

FIM

16/09/2026, 14:03:14

RESULTADO E OBSERVACOES

8 de 8 testes unitários de aceitação passaram com sucesso (100%). Codificação manual.

ARQUIVOS ANEXADOS

3 arquivo(s) de código ou texto anexado(s) ao trial.

- texto-do-trial.txt (166 linhas)
- TarifadorHospedagem.java (128 linhas)
- CategoriaQuarto.java (26 linhas)

texto-do-trial.txt

Participante: Luis Henrique Gonçalves Barbosa
Kata: Tarifador de Hospedagem e Pousada
Tratamento: SEM_IA
Tempo: 25:30

Código implementado:

```
// === CategoriaQuarto.java ===
package br.ufc.lab02.hospedagem;

import java.math.BigDecimal;

public enum CategoriaQuarto {
    STANDARD(1.00, BigDecimal.valueOf(50.00)),
    LUXO(1.30, BigDecimal.valueOf(80.00)),
    PRESIDENCIAL(1.70, BigDecimal.valueOf(120.00));

    private final double fatorMultiplicador;
    private final BigDecimal taxaLimpeza;

    CategoriaQuarto(double fatorMultiplicador, BigDecimal taxaLimpeza) {
        this.fatorMultiplicador = fatorMultiplicador;
        this.taxaLimpeza = taxaLimpeza;
    }

    public double getFatorMultiplicador() {
        return fatorMultiplicador;
    }

    public BigDecimal getTaxaLimpeza() {
        return taxaLimpeza;
    }
}

// === TarifadorHospedagem.java ===
package br.ufc.lab02.hospedagem;

import java.math.BigDecimal;
import java.math.RoundingMode;
import java.time.LocalDate;
import java.time.temporal.ChronoUnit;

public class TarifadorHospedagem {

    private static final BigDecimal TARIFA_CURTA = BigDecimal.valueOf(160.00);
    private static final BigDecimal TARIFA_MEDIA = BigDecimal.valueOf(130.00);
    private static final BigDecimal TARIFA_LONGA = BigDecimal.valueOf(100.00);
    private static final BigDecimal VALOR_HOSPEDE_EXTRA = BigDecimal.valueOf(40.00);
    private static final BigDecimal FATOR_DESCONTO_CONVENIO = BigDecimal.valueOf(0.90);

    public BigDecimal calcularHospedagem(
        LocalDate checkIn,
        LocalDate checkOut,
        int numeroHospedes,
        CategoriaQuarto categoria,
        boolean possuiConvenio) {
```

```

        validarReserva(checkIn, checkOut, numeroHospedes, categoria);

        long totalDiarias = calcularQuantidadeDeDiarias(checkIn, checkOut);
        BigDecimal tarifaDiaria = selecionarTarifaDiaria(totalDiarias);
        BigDecimal custoDasDiarias = calcularCustoDasDiarias(tarifaDiaria, totalDiarias);
        BigDecimal custoDosHospedesExtras = calcularCustoDosHospedesExtras(
            numeroHospedes,
            totalDiarias
        );

        BigDecimal subtotal = custoDasDiarias.add(custoDosHospedesExtras);
        BigDecimal subtotalComCategoria = aplicarCategoria(subtotal, categoria);
        BigDecimal totalComLimpeza = adicionarTaxaDeLimpeza(subtotalComCategoria,
categoria);
        BigDecimal totalComConvenio = aplicarConvenio(totalComLimpeza, possuiConvenio);

        return formatarValor(totalComConvenio);
    }

    private void validarReserva(
        LocalDate checkIn,
        LocalDate checkOut,
        int numeroHospedes,
        CategoriaQuarto categoria) {

        if (checkIn == null) {
            throw new IllegalArgumentException("Datas de check-in e check-out sao
obrigatorias.");
        }

        if (checkOut == null) {
            throw new IllegalArgumentException("Datas de check-in e check-out sao
obrigatorias.");
        }

        if (!checkOut.isAfter(checkIn)) {
            throw new IllegalArgumentException("A data de check-out deve ser posterior ao
check-in.");
        }

        if (numeroHospedes <= 0) {
            throw new IllegalArgumentException("Numero de hospedes deve ser entre 1 e
5.");
        }

        if (numeroHospedes > 5) {
            throw new IllegalArgumentException("Numero de hospedes deve ser entre 1 e
5.");
        }

        if (categoria == null) {
            throw new IllegalArgumentException("Categoria do quarto e obrigatoria.");
        }
    }

    private long calcularQuantidadeDeDiarias(LocalDate checkIn, LocalDate checkOut) {
        return ChronoUnit.DAYS.between(checkIn, checkOut);
    }

    private BigDecimal selecionarTarifaDiaria(long totalDiarias) {
        if (totalDiarias <= 3) {

```

```

        return TARIFA_CURTA;
    }

    if (totalDiarias <= 7) {
        return TARIFA_MEDIA;
    }

    return TARIFA_LONGA;
}

private BigDecimal calcularCustoDasDiarias(BigDecimal tarifaDiaria, long
totalDiarias) {
    BigDecimal quantidadeDeDiarias = BigDecimal.valueOf(totalDiarias);
    return tarifaDiaria.multiply(quantidadeDeDiarias);
}

private BigDecimal calcularCustoDosHospedesExtras(int numeroHospedes, long
totalDiarias) {
    if (numeroHospedes <= 2) {
        return BigDecimal.ZERO;
    }

    int quantidadeDeHospedesExtras = numeroHospedes - 2;
    BigDecimal quantidadeDeExtras = BigDecimal.valueOf(quantidadeDeHospedesExtras);
    BigDecimal quantidadeDeDiarias = BigDecimal.valueOf(totalDiarias);

    BigDecimal adicionalPorDiaria = VALOR_HOSPEDE_EXTRA.multiply(quantidadeDeExtras);
    return adicionalPorDiaria.multiply(quantidadeDeDiarias);
}

private BigDecimal aplicarCategoria(BigDecimal subtotal, CategoriaQuarto categoria) {
    BigDecimal fatorDaCategoria =
BigDecimal.valueOf(categoria.getFatorMultiplicador());
    return subtotal.multiply(fatorDaCategoria);
}

private BigDecimal adicionarTaxaDeLimpeza(BigDecimal subtotal, CategoriaQuarto
categoria) {
    BigDecimal taxaDeLimpeza = categoria.getTaxaLimpeza();
    return subtotal.add(taxaDeLimpeza);
}

private BigDecimal aplicarConvenio(BigDecimal valor, boolean possuiConvenio) {
    if (possuiConvenio) {
        return valor.multiply(FATOR_DESCONTO_CONVENIO);
    }

    return valor;
}

private BigDecimal formatarValor(BigDecimal valor) {
    return valor.setScale(2, RoundingMode.HALF_EVEN);
}
}

```

TarifadorHospedagem.java

```
package br.ufc.lab02.hospedagem;

import java.math.BigDecimal;
import java.math.RoundingMode;
import java.time.LocalDate;
import java.time.temporal.ChronoUnit;

public class TarifadorHospedagem {

    private static final BigDecimal TARIFA_CURTA = BigDecimal.valueOf(160.00);
    private static final BigDecimal TARIFA_MEDIA = BigDecimal.valueOf(130.00);
    private static final BigDecimal TARIFA_LONGA = BigDecimal.valueOf(100.00);
    private static final BigDecimal VALOR_HOSPEDE_EXTRA = BigDecimal.valueOf(40.00);
    private static final BigDecimal FATOR_DESCONTO_CONVENIO = BigDecimal.valueOf(0.90);

    public BigDecimal calcularHospedagem(
        LocalDate checkIn,
        LocalDate checkOut,
        int numeroHospedes,
        CategoriaQuarto categoria,
        boolean possuiConvenio) {

        validarReserva(checkIn, checkOut, numeroHospedes, categoria);

        long totalDiarias = calcularQuantidadeDeDiarias(checkIn, checkOut);
        BigDecimal tarifaDiaria = selecionarTarifaDiaria(totalDiarias);
        BigDecimal custoDasDiarias = calcularCustoDasDiarias(tarifaDiaria, totalDiarias);
        BigDecimal custoDosHospedesExtras = calcularCustoDosHospedesExtras(
            numeroHospedes,
            totalDiarias
        );

        BigDecimal subtotal = custoDasDiarias.add(custoDosHospedesExtras);
        BigDecimal subtotalComCategoria = aplicarCategoria(subtotal, categoria);
        BigDecimal totalComLimpeza = adicionarTaxaDeLimpeza(subtotalComCategoria,
            categoria);
        BigDecimal totalComConvenio = aplicarConvenio(totalComLimpeza, possuiConvenio);

        return formatarValor(totalComConvenio);
    }

    private void validarReserva(
        LocalDate checkIn,
        LocalDate checkOut,
        int numeroHospedes,
        CategoriaQuarto categoria) {

        if (checkIn == null) {
            throw new IllegalArgumentException("Datas de check-in e check-out sao obrigatorias.");
        }

        if (checkOut == null) {
            throw new IllegalArgumentException("Datas de check-in e check-out sao obrigatorias.");
        }

        if (!checkOut.isAfter(checkIn)) {

```

```

        throw new IllegalArgumentException("A data de check-out deve ser posterior ao
check-in.");
    }

    if (numeroHospedes <= 0) {
        throw new IllegalArgumentException("Numero de hospedes deve ser entre 1 e
5.");
    }

    if (numeroHospedes > 5) {
        throw new IllegalArgumentException("Numero de hospedes deve ser entre 1 e
5.");
    }

    if (categoria == null) {
        throw new IllegalArgumentException("Categoria do quarto e obrigatoria.");
    }
}

private long calcularQuantidadeDeDiarias(LocalDate checkIn, LocalDate checkOut) {
    return ChronoUnit.DAYS.between(checkIn, checkOut);
}

private BigDecimal selecionarTarifaDiaria(long totalDiarias) {
    if (totalDiarias <= 3) {
        return TARIFA_CURTA;
    }

    if (totalDiarias <= 7) {
        return TARIFA_MEDIA;
    }

    return TARIFA_LONGA;
}

private BigDecimal calcularCustoDasDiarias(BigDecimal tarifaDiaria, long
totalDiarias) {
    BigDecimal quantidadeDeDiarias = BigDecimal.valueOf(totalDiarias);
    return tarifaDiaria.multiply(quantidadeDeDiarias);
}

private BigDecimal calcularCustoDosHospedesExtras(int numeroHospedes, long
totalDiarias) {
    if (numeroHospedes <= 2) {
        return BigDecimal.ZERO;
    }

    int quantidadeDeHospedesExtras = numeroHospedes - 2;
    BigDecimal quantidadeDeExtras = BigDecimal.valueOf(quantidadeDeHospedesExtras);
    BigDecimal quantidadeDeDiarias = BigDecimal.valueOf(totalDiarias);

    BigDecimal adicionalPorDiaria = VALOR_HOSPEDE_EXTRA.multiply(quantidadeDeExtras);
    return adicionalPorDiaria.multiply(quantidadeDeDiarias);
}

private BigDecimal aplicarCategoria(BigDecimal subtotal, CategoriaQuarto categoria) {
    BigDecimal fatorDaCategoria =
BigDecimal.valueOf(categoria.getFatorMultiplicador());
    return subtotal.multiply(fatorDaCategoria);
}

```

```

        private BigDecimal adicionarTaxaDeLimpeza(BigDecimal subtotal, CategoriaQuarto
categoria) {
            BigDecimal taxaDeLimpeza = categoria.getTaxaLimpeza();
            return subtotal.add(taxaDeLimpeza);
        }

        private BigDecimal aplicarConvenio(BigDecimal valor, boolean possuiConvenio) {
            if (possuiConvenio) {
                return valor.multiply(FATOR_DESCONTO_CONVENIO);
            }

            return valor;
        }

        private BigDecimal formatarValor(BigDecimal valor) {
            return valor.setScale(2, RoundingMode.HALF_EVEN);
        }
    }

```

CategoriaQuarto.java

```
package br.ufc.lab02.hospedagem;

import java.math.BigDecimal;

public enum CategoriaQuarto {
    STANDARD(1.00, BigDecimal.valueOf(50.00)),
    LUXO(1.30, BigDecimal.valueOf(80.00)),
    PRESIDENCIAL(1.70, BigDecimal.valueOf(120.00));

    private final double fatorMultiplicador;
    private final BigDecimal taxaLimpeza;

    CategoriaQuarto(double fatorMultiplicador, BigDecimal taxaLimpeza) {
        this.fatorMultiplicador = fatorMultiplicador;
        this.taxaLimpeza = taxaLimpeza;
    }

    public double getFatorMultiplicador() {
        return fatorMultiplicador;
    }

    public BigDecimal getTaxaLimpeza() {
        return taxaLimpeza;
    }
}
```